

Práctica 1:

Búsqueda y Satisfacción de Restricciones

Sistemas Inteligentes



ÍNDICE

1. INTRODUCCIÓN	2
2. IMPLEMENTACIÓN DE ALGORITMOS	3
2.1. BACKTRACKING	3
2.1.1. MEJORAS IMPLEMENTADAS	4
2.2. CONSISTENCIA DE ARCO (AC3)	5
2.2.1. ERRORES Y MEJORAS IMPLEMENTADAS	6
2.3. FORWARD CHECKING	8
2.3.1. ERRORES Y MEJORAS IMPLEMENTADAS	10
3. COMPARACIÓN DE RENDIMIENTO ENTRE ALGORITMOS	13
3.1. COMPARACIÓN EN LOS TIEMPOS DE EJECUCIÓN	13
3.2. COMPARACIÓN EN LA CANTIDAD DE SALTOS	15
4. CONCLUSIONES	16
5. BIBLIOGRAFÍA	18

1. INTRODUCCIÓN

El objetivo de este informe es introducirse en los sistemas inteligentes a través de la aplicación de técnicas de inteligencia artificial para resolver un problema clásico, el Sudoku. En particular, se enfocará en la búsqueda inteligente combinada con la satisfacción de restricciones, dos pilares fundamentales de la inteligencia artificial, que tienen como objetivo optimizar el rendimiento computacional y reducir la memoria utilizada, evitando cálculos innecesarios y almacenamiento excesivo de datos.

La búsqueda inteligente implica el uso de algoritmos que exploran de manera eficiente un conjunto de soluciones en busca de la óptima. Por otro lado, la satisfacción de restricciones es una técnica que se centra en encontrar una solución que satisfaga un conjunto de restricciones definidas para el problema.

En el contexto del Sudoku, esto implica encontrar una asignación de números que cumplan con restricciones tales como no repetir el mismo número en una fila o en una columna del tablero 9x9. A la vez que tampoco se pueden repetir un número en los subtableros 3x3 que componen al tablero principal.

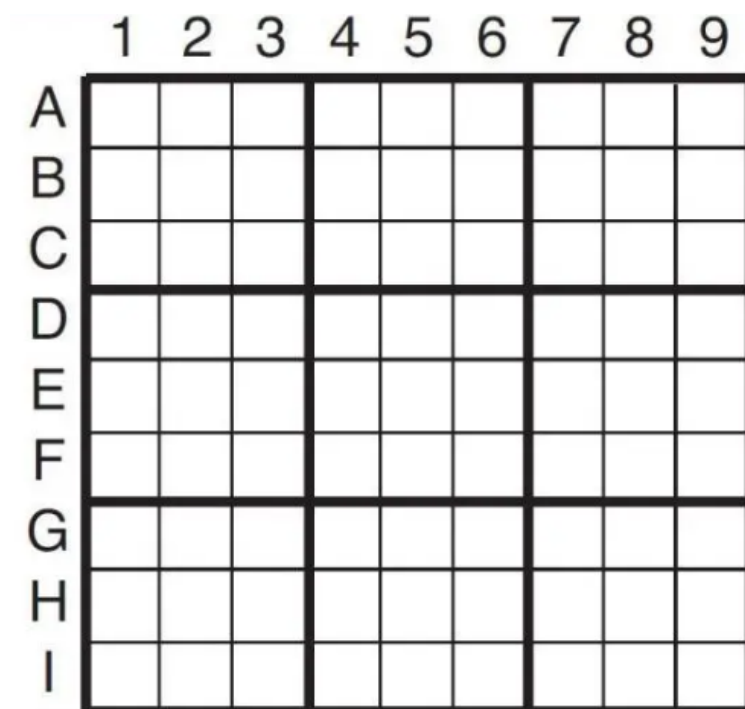


Imagen 1. Tablero de Sudoku

Para la elaboración de los algoritmos propuestos, es fundamental comprender las restricciones previamente citadas. Por otro lado, los algoritmos se desarrollarán específicamente en Python, aprovechando su versatilidad y facilidad de implementación respecto a otros lenguajes de programación, garantizará una ejecución eficiente y precisa.

2. IMPLEMENTACIÓN DE ALGORITMOS

2.1. BACKTRACKING

Backtracking es un algoritmo de programación que se utiliza para resolver problemas de forma recursiva, construyendo la solución paso a paso. En cada paso, se toma una decisión y se explora una rama del árbol de posibles soluciones. Si en algún momento se encuentra una situación en la que no se puede continuar sin infringir una restricción o sin llegar a una solución incorrecta, se retrocede y se prueba otra opción.

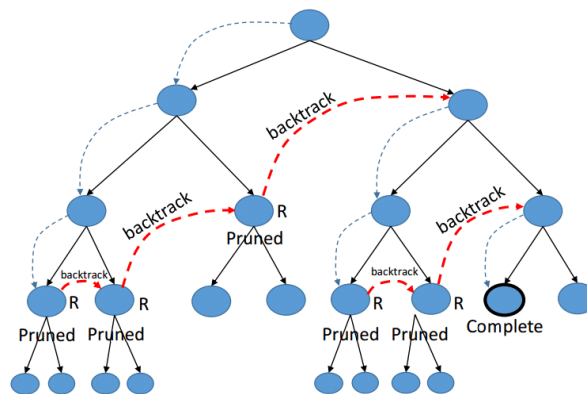


Imagen 2. Ejemplo del algoritmo de Backtracking

En el contexto de resolver un Sudoku, se comienza con un tablero parcialmente lleno y el objetivo es completarlo siguiendo todas las restricciones. Cada casilla se considera como un espacio en el que colocar un número válido, es decir, del 1 al 9. Si no se encuentra una solución válida, se retrocede y se prueba con un número diferente en la casilla anterior. Este proceso continúa recursivamente hasta que se complete el tablero o se llegue a que no hay una solución válida.[\[1\]](#)

En el caso de la práctica propuesta, se ha implementado una función recursiva cuyo caso base es que el tablero ya este completo, es decir, todas sus celdas estén ocupadas. En el caso opuesto, esta función deberá recorrer el tablero, desde la esquina superior izquierda hasta la esquina inferior derecha, hasta encontrar una casilla cuyo valor sea 0, es decir, este vacía.

Para cada celda vacía encontrada, la función prueba diferentes números, asegurándose de que cada número probado cumpla con las restricciones establecidas.

Si se encuentra un número válido, se coloca en la celda y se realiza una llamada recursiva a la función para continuar explorando el tablero.

Si se han probado todos los valores posibles para una casilla y ninguno cumple con las restricciones, significa que se ha llegado a un punto donde no se puede seguir avanzando más en esa rama del árbol de posibilidades. Esto se debe a que ninguna de las opciones disponibles es válida para el tablero y las restricciones dadas. Por lo tanto, se debe retroceder a la casilla anterior donde se realizó esa asignación incorrecta y probar con otro valor para esa casilla.

2.1.1. ERRORES Y MEJORAS IMPLEMENTADAS

Aunque el algoritmo cumplía con los requisitos solicitados y lograba resolver el Sudoku, no era del todo eficiente. Este se basaba en un enfoque de fuerza bruta, lo que significaba que probaba todas las combinaciones posibles sin considerar restricciones adicionales que podrían reducir el espacio de búsqueda.

Para abordar esta limitación, se implementaron los dominios, es decir, un conjunto de valores posibles que puede tomar una variable en el problema. Al utilizar dominios, se logra reducir el espacio de búsqueda y, en consecuencia, se obtiene un algoritmo más eficiente. Esto se debe a que automáticamente se descartan valores que no pueden formar parte de la solución, mejorando así el proceso de búsqueda y la resolución del Sudoku. [4]

Para implementar los dominios en el código, se creó una nueva clase que representaba una casilla del tablero de Sudoku. Esta clase contiene las siguientes variables asociadas a cada casilla:

- **Fila:** la fila en la que se encuentra la casilla.
- **Columna:** la columna en la que se encuentra la casilla.
- **Valor:** el valor actual de la casilla, si está asignada.
- **Dominio:** el conjunto de posibles valores que puede tomar la casilla.

Luego, se creó una matriz con las mismas dimensiones que el tablero, donde cada elemento de la matriz es una instancia de esta clase que representa una casilla del Sudoku. Así, cada casilla del tablero está asociada con una instancia de esta clase, lo que facilita el trabajo y la manipulación de las casillas. [4]

	1	2	3	4	5	6	7	8	9
A	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio
B	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio
C	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio
D	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio
E	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio
F	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio
G	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio
H	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio
I	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio	Fila Columna Valor Dominio

Imagen 3. Representación gráfica de la matriz de variables

2.2. CONSISTENCIA DE ARCO (AC3)

AC3 (Arc Consistency 3) es un algoritmo utilizado en la resolución de problemas de restricción para reducir el espacio de búsqueda eliminando valores inconsistentes de los dominios de las variables. Dicho de otro modo, AC3 se encarga de transformar problemas que en un principio pueden parecer complejos, en otros más sencillos al verificar si las restricciones entre las variables cumplen con la propiedad de consistencia de arcos.

Esta consistencia de arcos implica asegurar que todas las combinaciones posibles de valores entre las variables respeten las restricciones establecidas.

La relación entre AC3 y Backtracking es que son algoritmos complementarios, AC3 se encarga de disminuir significativamente la cantidad de ramas que el algoritmo de backtracking necesita explorar.[\[2\]](#)[\[3\]](#)

En el contexto del Sudoku, se ha seguido el pseudocódigo aportado en la práctica:

```
Q = conjunto de restricciones binarias <Vi, Vj> para cada arco (ep) en E, donde i ≠ j
Mientras Q no esté vacío hacer
    <Vk, Vm> = seleccionar y borrar un elemento de Q
    cambio = falso
    Para cada valor vk en el dominio Dk hacer
        Si no_consistente(vk, Dm) entonces
            borrar vk de Dk
            cambio = verdadero
    FinSi
FinPara
Si Dk está vacío entonces
    salir sin solución
FinSi
Si cambio es verdadero entonces
    Para cada arco (er) en E donde i ≠ k y i ≠ m hacer
        agregar restricción binaria <Vi, Vm> a Q
    FinPara
FinSi
FinMientras
```

Imagen 4. Pseudocódigo del AC3

A la hora de implementar este algoritmo, se comenzó por la generación de restricciones. Para ello, se tuvo en cuenta que había que recorrer el tablero como si de una matriz se tratara y almacenar, para la casilla actual, todas las celdas adyacentes que estuvieran en esa fila, columna o minitablero 3x3.

Para comprobar que se estaban obteniendo las restricciones correctamente se imprimieron de forma ordenada utilizando el comando '**aristas_ordenadas = sorted(aristas)**' siendo '**aristas_ordenadas**' el vector resultante y '**aristas**' el vector a ordenar. La ordenación garantiza que las restricciones se presenten de manera clara y organizada con el objetivo de detectar cualquier error en la implementación.

```
ntico@ubuntu:~/Escritorio/Inteligentes$ python3 main.py
Aristas generadas:
((0, 0), (0, 1))
((0, 0), (0, 2))
((0, 0), (0, 3))
((0, 0), (0, 4))
((0, 0), (0, 5))
((0, 0), (0, 6))
((0, 0), (0, 7))
((0, 0), (0, 8))
((0, 0), (1, 0))
((0, 0), (1, 1))
((0, 0), (1, 2))
((0, 0), (2, 0))
((0, 0), (2, 1))
((0, 0), (2, 2))
((0, 0), (3, 0))
((0, 0), (4, 0))
((0, 0), (5, 0))
((0, 0), (6, 0))
((0, 0), (7, 0))
((0, 0), (8, 0))
((0, 1), (0, 0))
((0, 1), (0, 2))
((0, 1), (0, 3))
((0, 1), (0, 4))
((0, 1), (0, 5))
((0, 1), (0, 6))
((0, 1), (0, 7))
```

Imagen 5. Primeras aristas de restricciones

Cada celda en un Sudoku está relacionada con 8 celdas en su fila, 8 en su columna y 4 más en su minitablero. Por lo tanto, hay un total de 20 restricciones por cada una de las 81 casilla del tablero, entonces, se deben obtener 1600 restricciones para todo el tablero.

Otro aspecto importante de la implementación ha sido el método para reducir los dominios. Este método se ha encargado de acceder a los dominios de las dos variables involucradas en la restricción. Por cada valor en el dominio de la primera variable, se verifica si existe un valor en el dominio de la segunda variable que sea igual. Si se encuentra un valor igual en ambos dominios, se elimina el valor del dominio de la primera variable, ya que esta asignación infringe una restricción del problema.

2.2.1. ERRORES Y MEJORAS IMPLEMENTADAS

Tras la implementación del algoritmo se puede ver que el Sudoku no se resolvía de la manera que se esperaba, a pesar de la implementación del algoritmo AC3. Es por ello por lo que se creó una traza que comparaba los dominios antes y después de ejecutar el AC3 con el objetivo de encontrar la raíz del problema.

Al observar que los dominios no sufrían cambios después de ejecutar el algoritmo AC3, se llegó a la conclusión de que los dominios modificados por AC3 se restablecían al ejecutar el algoritmo de Backtracking. Esto llevó a que había algo que impedía que los dominios modificados surtieran efecto.

Para resolver este problema, se introdujo una nueva variable booleana en la clase 'Solver'. Esta variable se encargaba de registrar si los dominios se habían calculado previamente por cualquiera de los dos métodos. Si los dominios ya se habían calculado, el método correspondiente no volvía a calcularlos, evitando así que se sobrescribieran los cambios realizados por el otro método.^[4]

Esta solución permitió evitar el solapamiento de los dominios y aseguró que los cambios introducidos por el algoritmo AC3 fueran efectivos en la resolución del Sudoku.

DOMINIOS INICIALES	
CELDA (0, 0) : {3}	CELDA (5, 0) : {6}
CELDA (0, 1) : {1}	CELDA (5, 1) : {2}
CELDA (0, 2) : {2}	CELDA (5, 2) : {9}
CELDA (0, 3) : {4}	CELDA (5, 3) : {5}
CELDA (0, 4) : {5, 7}	CELDA (5, 4) : {4}
CELDA (0, 5) : {5, 6}	CELDA (5, 5) : {1}
CELDA (0, 6) : {7, 9}	CELDA (5, 6) : {8}
CELDA (0, 7) : {7, 9}	CELDA (5, 7) : {7}
CELDA (0, 8) : {8}	CELDA (5, 8) : {3, 7}
CELDA (1, 0) : {7}	CELDA (6, 0) : {8}
CELDA (1, 1) : {4}	CELDA (6, 1) : {5, 9}
CELDA (1, 2) : {5, 8}	CELDA (6, 2) : {1}
CELDA (1, 3) : {2}	CELDA (6, 3) : {7, 9}
CELDA (1, 4) : {9}	CELDA (6, 4) : {2}
CELDA (1, 5) : {3}	CELDA (6, 5) : {5, 9}
CELDA (1, 6) : {6}	CELDA (6, 6) : {4, 7}
CELDA (1, 7) : {1, 2}	CELDA (6, 7) : {3}
CELDA (1, 8) : {1, 2, 5}	CELDA (6, 8) : {6, 7}
CELDA (2, 0) : {9}	CELDA (7, 0) : {4, 9}
CELDA (2, 1) : {6}	CELDA (7, 1) : {3}
CELDA (2, 2) : {5}	CELDA (7, 2) : {6}
CELDA (2, 3) : {1}	CELDA (7, 3) : {8}
CELDA (2, 4) : {5, 7}	CELDA (7, 4) : {1}
CELDA (2, 5) : {8}	CELDA (7, 5) : {9}
CELDA (2, 6) : {2, 3, 4, 7, 9}	CELDA (7, 6) : {5}
CELDA (2, 7) : {2, 4, 7, 9}	CELDA (7, 7) : {2, 4, 7}
CELDA (2, 8) : {2, 3, 5, 7}	CELDA (7, 8) : {2, 7}
CELDA (3, 0) : {1}	CELDA (8, 0) : {2}
CELDA (3, 1) : {8}	CELDA (8, 1) : {5}
CELDA (3, 2) : {3}	CELDA (8, 2) : {5, 7}
CELDA (3, 3) : {9}	CELDA (8, 3) : {6}
CELDA (3, 4) : {6}	CELDA (8, 4) : {3, 5, 7}
CELDA (3, 5) : {7}	CELDA (8, 5) : {4}
CELDA (3, 6) : {2, 3, 9}	CELDA (8, 6) : {1}
CELDA (3, 7) : {5}	CELDA (8, 7) : {7, 8}
CELDA (3, 8) : {4}	CELDA (8, 8) : {9}
CELDA (4, 0) : {5}	
CELDA (4, 1) : {7}	
CELDA (4, 2) : {4}	
CELDA (4, 3) : {3}	
CELDA (4, 4) : {8}	
CELDA (4, 5) : {2}	
CELDA (4, 6) : {9}	
CELDA (4, 7) : {6}	
CELDA (4, 8) : {1}	

Imagen 6. Dominios antes del AC3 para el SUDOKU 1

DOMINIOS AC3	
CELDA (0, 0) : {3}	CELDA (5, 0) : {6}
CELDA (0, 1) : {1}	CELDA (5, 1) : {2}
CELDA (0, 2) : {2}	CELDA (5, 2) : {9}
CELDA (0, 3) : {4}	CELDA (5, 3) : {5}
CELDA (0, 4) : {5}	CELDA (5, 4) : {4}
CELDA (0, 5) : {6}	CELDA (5, 5) : {1}
CELDA (0, 6) : {7}	CELDA (5, 6) : {8}
CELDA (0, 7) : {9}	CELDA (5, 7) : {7}
CELDA (0, 8) : {8}	CELDA (5, 8) : {3}
CELDA (1, 0) : {7}	CELDA (6, 0) : {8}
CELDA (1, 1) : {4}	CELDA (6, 1) : {9}
CELDA (1, 2) : {8}	CELDA (6, 2) : {1}
CELDA (1, 3) : {2}	CELDA (6, 3) : {7}
CELDA (1, 4) : {9}	CELDA (6, 4) : {2}
CELDA (1, 5) : {3}	CELDA (6, 5) : {5}
CELDA (1, 6) : {6}	CELDA (6, 6) : {4}
CELDA (1, 7) : {1}	CELDA (6, 7) : {3}
CELDA (1, 8) : {5}	CELDA (6, 8) : {6}
CELDA (2, 0) : {9}	CELDA (7, 0) : {4}
CELDA (2, 1) : {6}	CELDA (7, 1) : {3}
CELDA (2, 2) : {5}	CELDA (7, 2) : {6}
CELDA (2, 3) : {1}	CELDA (7, 3) : {8}
CELDA (2, 4) : {7}	CELDA (7, 4) : {1}
CELDA (2, 5) : {8}	CELDA (7, 5) : {9}
CELDA (2, 6) : {3}	CELDA (7, 6) : {5}
CELDA (2, 7) : {4}	CELDA (7, 7) : {2}
CELDA (2, 8) : {2}	CELDA (7, 8) : {7}
CELDA (3, 0) : {1}	CELDA (8, 0) : {2}
CELDA (3, 1) : {8}	CELDA (8, 1) : {5}
CELDA (3, 2) : {3}	CELDA (8, 2) : {7}
CELDA (3, 3) : {9}	CELDA (8, 3) : {6}
CELDA (3, 4) : {6}	CELDA (8, 4) : {3}
CELDA (3, 5) : {7}	CELDA (8, 5) : {4}
CELDA (3, 6) : {2}	CELDA (8, 6) : {1}
CELDA (3, 7) : {5}	CELDA (8, 7) : {8}
CELDA (3, 8) : {4}	CELDA (8, 8) : {9}
CELDA (4, 0) : {5}	
CELDA (4, 1) : {7}	
CELDA (4, 2) : {4}	
CELDA (4, 3) : {3}	
CELDA (4, 4) : {8}	
CELDA (4, 5) : {2}	
CELDA (4, 6) : {9}	
CELDA (4, 7) : {6}	
CELDA (4, 8) : {1}	

Imagen 7. Dominios después del AC3 para el SUDOKU 1

Por otro lado, en un primer momento se generaron más aristas de las esperadas, ya que cada celda se consideraba vecina de sí misma. Para resolver este problema, se agregó un filtro para excluir la propia casilla de la lista de vecinos. De esta manera, al generar las restricciones se aseguraba que las restricciones entre las celdas fueran precisas, evitando la inclusión innecesaria de las celdas como sus propios vecinos.

2.3. FORWARD CHECKING

El algoritmo de Forward Checking (FC) realiza en cada paso de la búsqueda una verificación hacia delante de la asignación actual con respecto a todas las variables futuras que están restringidas por la variable actual.

Este proceso implica, por un lado, examinar todos los posibles valores de las variables futuras que están vinculadas con la variable actual y, por otro lado, que cualquier valor de las variables futuras que sea incompatible con la asignación actual se elimina temporalmente de sus dominios correspondientes.

Si durante este proceso el dominio de alguna variable futura queda vacío, significa que la asignación actual es inviable en términos de las restricciones del problema. En este caso, se retrocede y se deshace la instanciación de la variable actual.

Si ninguna de las instanciaciones anteriores es compatible con las restricciones, se procede con el backtracking cronológico, explorando diferentes valores para la variable actual en un intento de encontrar una solución válida. Este proceso se repite hasta que se encuentra una solución satisfactoria o se determina que no existe ninguna solución posible.[\[2\]](#)[\[3\]](#)

Por otro lado, la combinación de este algoritmo con AC3 puede proporcionar una estrategia aún más robusta. AC3 puede utilizarse para establecer la consistencia inicial del Sudoku y, tras esto, Forward Checking se aplica en cada paso de la búsqueda para mantener esa consistencia a medida que se realizan las asignaciones.

En el contexto del Sudoku, Forward Checking permite una búsqueda más eficiente de soluciones al anticipar y eliminar posibles conflictos, lo que conduce a una resolución más rápida y menos intensiva computacionalmente.

Para la elaboración del código, se ha seguido el siguiente pseudocódigo:

<pre>funcion FC(i: variable): booleano para cada a en dominio[i] hacer Xi ← a si i=N solución retorna CIERTO sino si forward (i,a) si FC(i+1) retorna CIERTO Restaurar (i) retorna FALSO procedimiento restaura(i: variable) para todo j=i+1 hasta N hacer para todo b en podado[j] hacer si Xi responsable filtrado b Eliminar b de podado[j] Añadir b a dominio[j]</pre>	<pre>funcion forward(i: variable, a: valor): booleano para todo j=i+1 hasta N hacer Vacio ← CIERTO para cada b en dominio[j] hacer si (a,b) está en Rij vacio ← FALSO sino eliminar b de dominio[j] añadir b a podado[j] si vacio retorna FALSO retorna CIERTO</pre>
---	---

Imagen 8. Pseudocódigo Forward Checking recursivo

En este, la función **FC** es la función principal del algoritmo, encargada de realizar la búsqueda recursiva tomando como entrada el índice de la variable actúan en el Sudoku, devolviendo un valor booleano que indica si se encontró solución o no.

Por otro lado, la función **restaura** se encarga de restaurar el estado de las variables después de que se haya probado un valor y no se haya encontrado una solución válida.

Por último, la función **forward** se encarga de verificar si es posible asignar el valor a la variable actual y continuar hacia delante manteniendo la consistencia con las

restricciones. Además, comprueba que el dominio de una variable siga teniendo valores, en caso contrario, significa que no hay una solución válida y la función retorna falso.

2.3.1. ERRORES Y MEJORAS IMPLEMENTADAS

A pesar de haber implementado las funciones anteriormente mencionadas al código principal, el algoritmo era incapaz de resolver correctamente algunos Sudokus ya que dejaba en blanco algunas casillas del tablero o proporcionaba soluciones incorrectas.

Es por ello por lo que, para entender mejor el comportamiento del algoritmo que se había implementado, se decidió reducir el problema a un Sudoku de tamaño 4x4 con el objetivo de simplificar el campo de errores y facilitar la identificación de errores.

En un primer momento se realizó una traza para saber qué pasos debía seguir exactamente el algoritmo:

	1		
			2
3		4	

Ilustración 1. Sudoku 4x4 propuesto

DOMINIOS INICIALES

$(0,0) : \{2, 4\}$ **$(0,1) : \{1\}$** $(0,2) : \{3\}$ $(0,3) : \{3, 4\}$
 $(1,0) : \{4\}$ $(1,1) : \{3, 4\}$ $(1,2) : \{1, 3\}$ **$(1,3) : \{2\}$**
 $(2,0) : \{1, 2, 4\}$ $(2,1) : \{2, 4\}$ $(2,2) : \{1, 2, 3\}$ $(2,3) : \{1, 3\}$
 $(3,0) : \{3\}$ $(3,1) : \{2\}$ **$(3,2) : \{4\}$** $(3,3) : \{1\}$

Al iniciar el algoritmo, se asigna el primer valor del dominio de la primera casilla vacía, en este caso, la casilla (0,0) y el valor 2. Al asignar ese valor, es necesario eliminar de los dominios de las casillas vecinas el valor que infrinja las restricciones y guardando como responsable la casilla origen, es decir:

2 {2, 4}	1 {1}	{3}	{3,4}
{4}	{3, 4}	{1, 3}	2 {2}
{1,4}	{2, 4}	{1,2,3}	{1,3}
3 {3}	{2}	4 {4}	{1}

Ahora, se asigna el valor 3 en la casilla (0,2) y se debe podar de la casilla (0,3), (1,2) y (2,2), guardando como responsable la (0,2)

2 {2, 4}	1 {1}	3 {3}	 {4}
 {4}	 {3, 4}	 {1}	2 {2}
 {1, 4}	 {2, 4}	 {1,2}	 {1,3}
3 {3}	 {2}	4 {4}	 {1}

Se asigna el 4 en la (0,3) y como no infringe restricciones asigna también el 4 en la (1,0) podándolo de la (1,1) y la (1,2) y guardando como responsable la (1,0):

2 {2}	1 {1}	3 {3}	4 {4}
4 {4}	 {3}	 {1}	2 {2}
 {1}	 {2, 4}	 {1,2}	 {1,3}
3 {3}	 {2}	4 {4}	 {1}

Ahora, se asigna el 3 en la (1,1) y como no infringe restricciones asigna también el 1 en la (1,2) podándolo de la (2,2) y guardando como responsable la (1,2):

2 {2}	1 {1}	3 {3}	4 {4}
4 {4}	3 {3}	1 {1}	2 {2}
 {1}	 {2, 4}	 {2}	 {1,3}
3 {3}	 {2}	4 {4}	 {1}

Se asigna el valor 1 en la casilla (2,0) y se debe podar de la casilla (2,3), guardando como responsable la (2,0):

2 {2}	1 {1}	3 {3}	4 {4}
4 {4}	3 {3}	1 {1}	2 {2}
1 {1}	{2, 4}	{2}	{3}
3 {3}	{2}	4 {4}	{1}

Ahora, se asigna el valor 2 en la (2,1) podándolo de la (2,2) y guardando como responsable la (2,1)

2 {2}	1 {1}	3 {3}	4 {4}
4 {4}	3 {3}	1 {1}	2 {2}
1 {1}	2 {2, 4}	{VACIO}	{3}
3 {3}	{2}	4 {4}	{1}

Sin embargo, como el dominio de la casilla (2,2) se queda vacío, es necesario restaurar el valor 2 de esa misma casilla y probar con otro valor en la casilla (2,1). Por tanto, se asigna el valor 4 en la (2,1)

2 {2}	1 {1}	3 {3}	4 {4}
4 {4}	3 {3}	1 {1}	2 {2}
1 {1}	4 {4}	{2}	{3}
3 {3}	{2}	4 {4}	{1}

Llegados a este punto, los valores restantes tienen asignaciones directas sin incumplir ninguna restricción, es decir, se asigna el valor 2 a la (2,2), el valor 3 a la (2,3), el 3 a la (3,0), el 2 a la (3,1), el 4 a la (3,2) y el 1 a la (3,3), completando así el sudoku de manera satisfactoria.

2 {2}	1 {1}	3 {3}	4 {4}
4 {4}	3 {3}	1 {1}	2 {2}
1 {1}	4 {4}	2 {2}	3 {3}
3 {3}	2 {2}	4 {4}	1 {1}

Una vez estaba claro el funcionamiento del algoritmo, se pudo ver que el error estaba en la función de restaurar puesto que esta recorría el tablero entero, es decir, desde la esquina superior izquierda hasta la esquina inferior derecha, lo cual no era del todo eficiente y era incorrecto para el algoritmo Forward Checking puesto que sólo debe mirar hacia delante. Para solucionar esto, se añadieron unas condiciones a modo de filtro para que, dados una fila y una columna, el tablero se recorriese a partir de ahí. [4]

3. COMPARACIÓN DE RENDIMIENTO ENTRE ALGORITMOS

Tras la implementación y verificación de los códigos con las mejoras realizadas y los errores corregidos, se ha comparado el rendimiento de cada uno de ellos para resolver los Sudokus propuestos. Esta evaluación ayudará a identificar el algoritmo más adecuado para diferentes tipos de Sudoku y a comprender de una forma más segura la finalidad de los algoritmos.

3.1. COMPARACIÓN EN LOS TIEMPOS DE EJECUCIÓN

Diferenciar los tiempos de ejecución permite evaluar y seleccionar los algoritmos más eficientes para resolver problemas de manera rápida y efectivo. Es por ello por lo que se han recopilado en la siguiente tabla y, para una mejor comprensión se han mostrado gráficamente.

	TIEMPOS DE EJECUCIÓN			
	BACKTRACKING	BT + AC3	FORWARD CHECKING	FC + AC3
SUDOKU 1	0,003	0,001	0,002	0,001
SUDOKU 2	0,014	0,001	0,019	0,001
SUDOKU 3	0,051	0,027	0,031	0,020
SUDOKU 4	0,046	0,043	0,021	0,016
SUDOKU 5	3,983	0,50	0,135	0,09
SUDOKU 6	0,050	0,049	0,024	0,021
SUDOKU 7	0,021	0,007	0,017	0,004
SUDOKU 8	5,317	4,198	1,741	1,172

Tabla 1. Diferencia entre los tiempos de ejecución

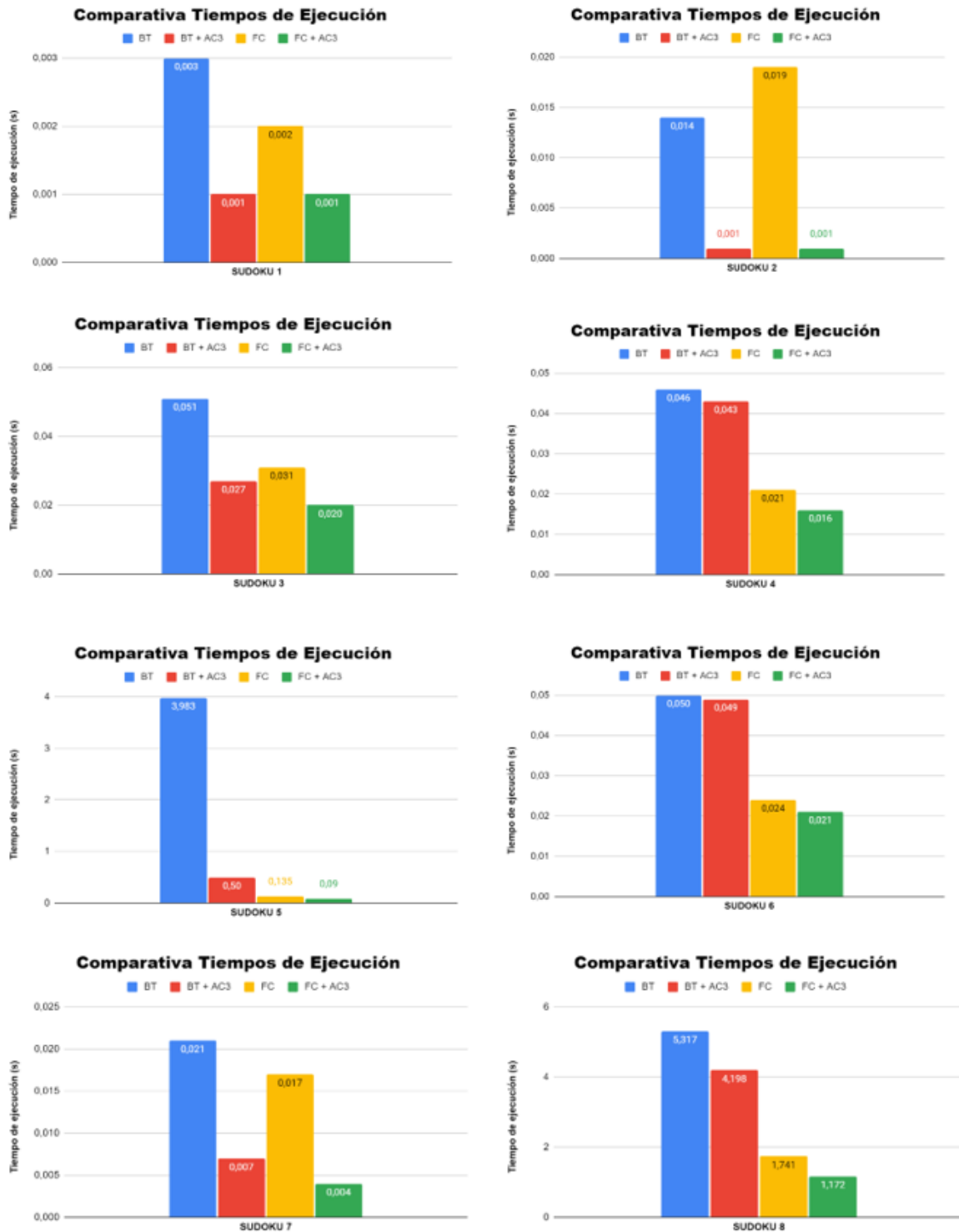


Ilustración 2. Gráficas que muestran los tiempos de ejecución

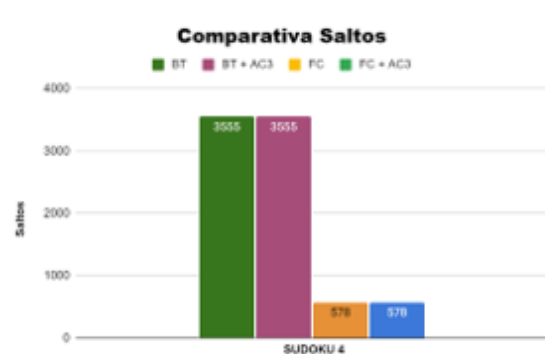
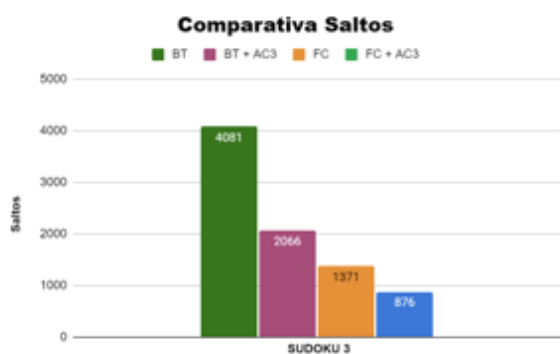
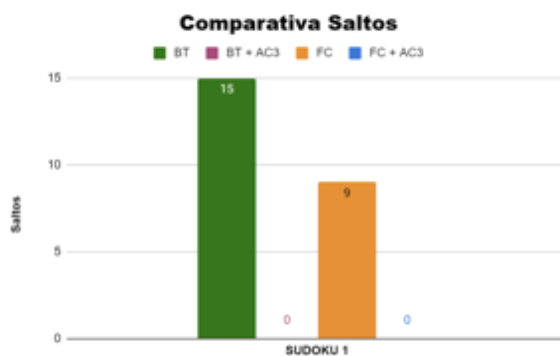
Como se puede observar, el algoritmo de Backtracking ha sido el que más ha tardado en ejecutarse, esto es debido a la cantidad de correcciones que debe ir haciendo a medida que se desarrolla. Por otro lado, se puede ver cómo el uso anticipado del AC3 junto con cualquier otro algoritmo ha reducido considerablemente los tiempos de ejecución en la mayoría de los sudokus. Por último, el Forward ha sido el algoritmo que menos ha tardado en todos los casos debido a que reduce el espacio de búsqueda y detecta conflictos con anticipación.

3.2. COMPARACIÓN EN LA CANTIDAD DE SALTOS

Por otro lado, se han diferenciado los saltos que ha necesitado dar cada algoritmo para completar el Sudoku de forma correcta. Esto es fundamental para obtener una idea del costo computacional que provoca cada algoritmo y, con esto, evaluar la eficiencia y efectividad en la resolución del problema planteado. Es por ello por lo que se han recopilado en la siguiente tabla y, para una mejor comprensión se han mostrado gráficamente.

	SALTOS ATRÁS			
	BACKTRACKING	BT + AC3	FORWARD CHECKING	FC + AC3
SUDOKU 1	15	0	9	0
SUDOKU 2	614	34	559	20
SUDOKU 3	4081	2066	1371	876
SUDOKU 4	3555	3555	578	578
SUDOKU 5	441458	52174	5268	4022
SUDOKU 6	3252	3252	602	602
SUDOKU 7	1322	298	466	192
SUDOKU 8	507183	399967	80392	56629

Tabla 2. Diferencia entre la cantidad de saltos



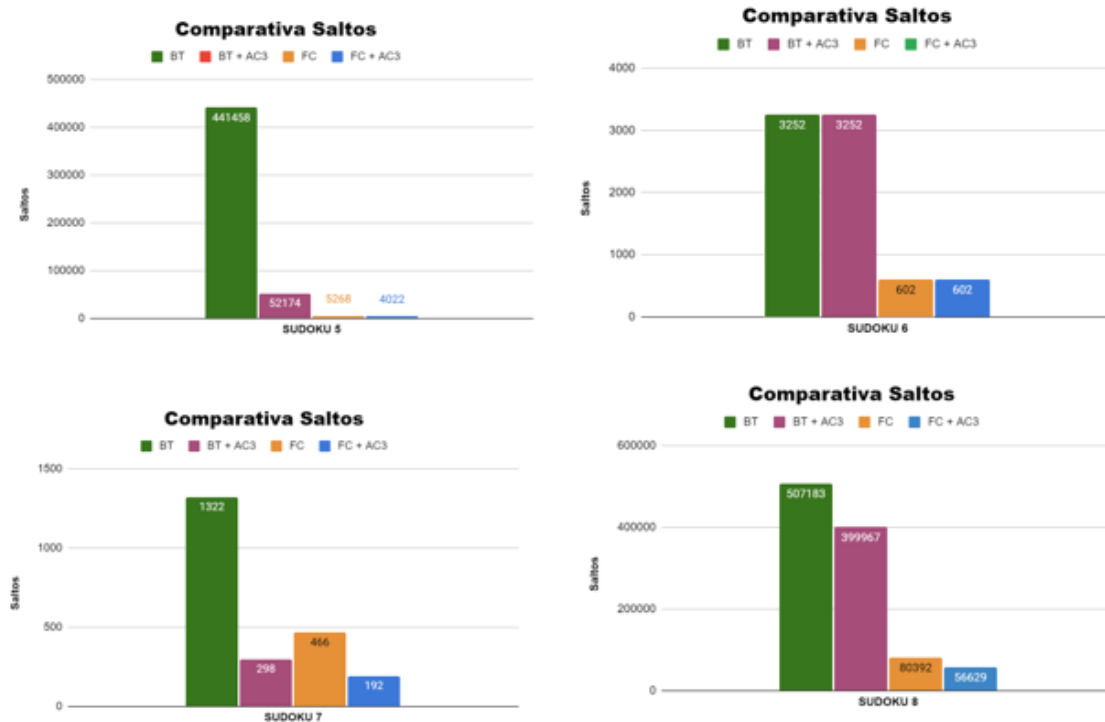


Ilustración 3. Gráficas que muestran la cantidad de saltos

Como se puede observar, el algoritmo de Backtracking ha tenido una cantidad de saltos considerablemente superior al resto, esto es debido a la cantidad de correcciones que debe ir haciendo a medida que se desarrolla lo que supone retroceder en el árbol de búsqueda. Por otro lado, se puede ver cómo el uso anticipado del AC3 junto con cualquier otro algoritmo ha reducido considerablemente los saltos dados ya que reduce los dominios y, por tanto, disminuye la necesidad de explorar múltiples ramas del árbol de búsqueda. Por último, el Forward ha sido el algoritmo que menos ha tardado en la mayoría de los casos debido a que reduce el espacio de búsqueda y detecta conflictos con anticipación.

4. CONCLUSIONES

A lo largo de la elaboración de la práctica, se han visto distintos algoritmos para la resolución de un problema de satisfacción de restricciones, en este caso el Sudoku.

Si bien es cierto que para la implementación de los algoritmos propuestos se ha utilizado como punto de partida los pseudocódigos proporcionados, se ha aprovechado también el uso de ChatGPT para obtener una estructura inicial del código. A partir de esta base, se ha procedido a desarrollar y adaptar el código final, realizando ajustes y optimizaciones según fuera necesario para desarrollar y completar los algoritmos.

En cuanto a los algoritmos, se ha podido observar que, a pesar de tener enfoques totalmente distintos, han sido capaz de resolver un mismo problema, AC3 ha sido capaz de simplificar el problema desde un inicio reduciendo considerablemente el espacio de búsqueda mientras que Backtracking, a pesar de ser más laborioso por la necesidad de corregir iterativamente, ha demostrado su utilidad en la exploración de todas las ramas del árbol de soluciones. Por otro lado, Forward Checking ha sido capaz de ser efectivo al reducir el espacio de búsqueda y detectar conflictos con anticipación.

Como se ha podido experimentar con diferentes tableros, se ha permitido observar las debilidades y fortalezas de cada uno de los algoritmos frente a un tablero parcialmente lleno de distintas formas. Además, se ha demostrado que el hecho de combinar algoritmos puede ser una manera de aumentar la efectividad de ciertos problemas puesto que se aprovechan las ventajas de cada uno de los algoritmos.

Por ello, es necesario destacar la efectividad del algoritmo AC3, capaz de simplificar considerablemente el problema, demostrando que elegir de forma adecuada ciertos algoritmos puede conducir a una solución correcta de forma más rápida.

En definitiva, este proyecto ha permitido comprender los distintos algoritmos y enfoques a los problemas de satisfacción de restricciones a la vez que su aplicación práctica en problemas del día a día tales como, resolver un simple Sudoku.



5. BIBLIOGRAFÍA

1. Chio Code. (2022, October 13). *Explicando backtracking | Fuerza bruta pero elegante* [Video]. YouTube. https://www.youtube.com/watch?v=ip2jC_kXGtg
2. Inteligencia Artificial. Un enfoque Moderno. Stuart Russell, Peter Noving. Ed. Prentice Hall. 2004
3. Artificial Intelligence. Eleaine Rich, Kevin Knight, ShivashankarB. Nair. Ed. McGraw-Hill. 2009
4. *ChatGPT*. (n.d.). <https://chat.openai.com/>